Algorithm Design and Implementation for Automated Timetable Scheduling

Muhammad Abdullah Khan Mahsud
Dept. of Computer Science
GIK Institute of Engineering Sciences and Technology
Topi, Pakistan
2023346@giki.edu.pk

Mujeeb U Rehman
Dept. of Computer Science
GIK Institute of Engineering Sciences and Technology
Topi, Pakistan
2023558@giki.edu.pk

Faizan Ali
Dept. of Computer Science
GIK Institute of Engineering Sciences and Technology
Topi, Pakistan
2023192@giki.edu.pk

***Abstract*—University scheduling is an old NP-hard combinatorial optimization problem. The size of courses, faculty, room counts, and student sections, grows exponentially, and brute-force enumeration is computationally infeasible at realistic scales. In this paper, the design and implementation of an automated timetable scheduler of GIK Institute of Engineering Sciences and Technology (GIKI), is introduced as a project developed during CS378 course Design and Analysis of Algorithms. The system formulates the scheduling problem as a Constraint Satisfaction Problem (CSP) and uses a backtracking search algorithm with two classical AI heuristics: Minimum Remaining Values (MRV) to order the variables and Forward Checking to detect early conflicts. Structured Excel files containing input data are read in using the pandas library, with an automatic credit-hour expansion step, which turns each credit hour into a single schedulable session. The solver searches the solver in a section-by-section and semester-by-semester manner to manage the search space. The results are presented as an interactive graphical user interface based on Tkinter and customized to display the results, with a combined weekly view, and exported to formally styled multi-page PDF timetables using ReportLab. The system was successful in scheduling more than 319 sessions in five departments – BCS, BDS, SE, BES and BAI – with zero hard constraint violation, which indicates the feasibility of classical algorithmic methods in the real world when it comes to institutional scheduling.**

***Index Terms*—constraint satisfaction problem, timetable scheduling, backtracking search, minimum remaining values, forward checking, NP-hard optimization**

I. Introduction

University course timetabling is among the most practically important and computationally challenging problems in academic administration. Each semester, scheduling coordinators must place hundreds of course sections into time slots and classrooms while simultaneously satisfying a substantial and sometimes-incompatible set of constraints: instructor availability, room capacity and type, departmental policies, section membership, and pedagogical preferences such as avoiding sequential courses for junior cohorts.

Timetabling in a manual form is slow, prone to errors, and lacks scalability with the growth of an institution. Computerized systems that impose constraint satisfaction are thus very practical. The formal NP-hardness of the problem [1], is the reason to use intelligent search techniques instead of brute-force search.

The project implements a timetabling system that is automated to timetable GIKI. The major contribution it brings is a Python CSP solver, which is an integration of backtracking search with MRV variable ordering and Forward Checking constraint propagation. The system also accommodates real world issues like building-routing constraints, shared-curriculum courses taught simultaneously to more than one section and soft preference constraints such as morning-slot preference during junior semesters and utilization of room capacity.

The rest of this paper is structured as follows: Section II states the problem formally; Section III describes the algorithm design; Section IV lists the assumptions; Section V discusses the constraint handling strategy; Section VI describes the input / output; Section VII and Section VIII analyze time and space complexity; Section IX discusses limitations; Section X presents results; and Section XI concludes.

II. Problem Definition

A. Entities

The scheduling problem is formulated over the following finite sets:

B. Credit-Hour Expansion

A course with credit value k is decomposed into k independent sessions. For example, a 3-credit lecture course produces three distinct scheduling variables, each requiring an independent assignment of a (room, time-slot) pair. This expansion is performed automatically during the Excel parsing stage.

TABLE I
PROBLEM ENTITIES

Symbol	Entity	Description
C	Courses	Course code, title, credit hours, type (lecture/lab), and assigned instructor(s).
T	Teachers	Instructors associated with one or more courses, possibly across multiple departments.
R	Rooms	Physical rooms characterized by capacity, type (lecture/lab), and building block.
S	Time-Slots	Discrete ordered (day, period) pairs spanning Monday–Friday.
Σ	Sections	Student sections grouped by department and semester (e.g., BCS-1A).

C. Formal CSP Formulation

Let V denote the set of all session variables generated by credit-hour expansion across all sections. Each variable $v \in V$ has a domain $D(v)$ – the set of feasible (room, time-slot) pairs satisfying basic type and building filters. The objective is to find a complete assignment $A : V \to \bigcup_v D(v)$ such that all hard constraints (Section V) are satisfied and soft constraint scoring is maximized.

D. Optimization Goal

The primary goal is completeness with respect to hard constraints. Secondary goals are encoded as soft constraints, including preferential morning-slot allocation for junior semesters and efficient room capacity utilization.

III. ALGORITHM DESIGN

A. Overview

A Backtracking Search algorithm is the heart of the solver. Consistency Backtracking is a methodical exploration of the assignment space variable by variable: at any point it picks an unsolved variable, and then iterates over candidate values (ranked according to a soft-constraint scoring). When a conflict is discovered, the algorithm switches back to the last point of choice and attempts the next candidate value.

B. Backtracking Search

The backtracking algorithm is developed according to the classical formulation of recursion:

1) Assuming that the current section/semester batch of variables is set, provide the entire assignment (success).
2) Choose an unassigned variable v with the help of the MRV heuristic (Section III-C).
3) To each value val $\in D(v)$ in increasing order of soft-constraint score (descending):
   a) When the value of val agrees with the existing partial assignment A (Section III-E), set $A(v) =$ val.
   b) Use Forward Checking to eliminate future variable domains (Section III-D).
   c) Recurrently call the backtracking process.
   d) When the recursive call is successful, pass the success upwards.
4) If no value in $D(v)$ yields a solution, return failure (backtrack).

The key advantage over naive generate-and-test is that upon detecting a violation in a partial assignment, the algorithm prunes entire subtrees of the search space without enumerating all complete assignments in that subtree.

C. Minimum Remaining Values (MRV) Heuristic

The MRV heuristic – also called the *fail-first* principle – selects at each choice point the unassigned variable with the smallest current domain:

$$v^* = \arg \min_{v \in \text{Unassigned}} |D(v)| \tag{1}$$

The intuition is that the variables with fewer remaining legal values are more prone to cause failure; thus addressing them in the initial stages would prevent the creation of large partial assignments that will eventually fail. The variables representing sessions whose instructor schedules are highly constrained or whose valid room/block combinations are scarce are natural occurrences in this scheduler that direct-search to the most critical bottlenecks first, guided by MRV. This always decreases the backtracks that can be seen in practice.

D. Forward Checking

The forward checking is implemented as soon as a tentative assignment $A(v) =$ val is made. In each future (unassigned) variable v' that shares a constraint with v, any element of $D(v')$ that is inconsistent with the new assignment value v is temporarily eliminated from $D(v')$.

If the domain of any future variable $D(v')$ becomes empty, the current assignment val is identified as a dead end without requiring a recursive call. The algorithm backtracks immediately and restores all pruned values.

In this scheduler, instructor and room conflicts are symmetric and dense – many sessions share an instructor or compete for room types – so forward propagation eliminates large blocks of invalid candidates on downstream variables before exploration, resulting in a significant reduction in effective branching factor.

E. Consistency Check

A candidate assignment of session variable v to (room r, slot s) is consistent with the current partial assignment A when:

1) **Teacher Conflict:** No other session in A shares an instructor at slot s, unless the session is a shared-curriculum course offered to two or more sections simultaneously (Section V-A).
2) **Room Conflict:** No other session in A uses room r at slot s.
3) **Building-Routing:** Room r belongs to the allowed building block for the course type and department (Section V-A).
4) **Section Conflict:** No other session in A is already assigned slot s for section σ.

F. Partitioned Solving Strategy

Constructing a monolithic CSP over all departments and sessions would yield a search space too large for real-time solving. The algorithm therefore employs a partitioned strategy:

- **Section-by-Section:** The solver processes one section (e.g., BCS-1A) at a time. Session variables are extracted, domains constructed, and the CSP solved independently.
- **Semester-by-Semester:** Departments are processed sequentially by semester, with coherent handling of shared-curriculum courses taught across multiple sections in the same semester.
- **Auto-Bucketing:** For shared-curriculum courses or semesters with many sessions, an auto-bucketing mechanism of 20 sessions per solve batch is applied. This bound limits branching and avoids worst-case exponential blowup, with each batch extended iteratively.

This disaggregated approach transforms a single intractable CSP into a sequence of bounded sub-problems, each solvable in under two seconds on commodity hardware.

IV. Assumptions

The assumptions below put the problem into context and maintain the system manageable:

1) **Fixed Time-Slot Grid:** The schedule is based on a regular weekly schedule (Monday–Friday, fixed intervals). Dynamic rescheduling and rolling calendars are not an option.
2) **Static Instructor Availability:** Instructor availability and course assignments are completely specified by the input Excel files. Dynamic change requests are not modeled.
3) **Negligible Inter-Block Travel Time:** Travel time between adjacent blocks of campus buildings (e.g., FCSE to AcB) is assumed negligible, and is not modeled as a constraint.
4) **Single-Room Sessions:** There is an assignment of one room per session. Multi-room or split-room designs are not supported.
5) **Section Homogeneity:** All the students in a section are supposed to be equally burdened by the course work as stated in the input of the curriculum. Exceptions at student level are not represented.
6) **Room Capacity as Soft Constraint:** Room capacity is modeled as a soft constraint (score-based preference) instead of a hard constraint to maximize solver completeness in limited room capacity.
7) **No Hard Consecutive-Session Constraint:** There are no hard constraints to consecutive sessions of any section; the scoring is mitigated by the soft preference in junior semesters to consecutive sessions.

V. Constraint Handling Strategy

A. Hard Constraints

Hard constraints are enforced by the consistency check invoked at every node of the search tree. An assignment is rejected when any hard constraint is violated.

H1 – Teacher Conflict Prevention: An instructor may not teach two sessions in the same time slot, with one exception: if a course is designated as a shared-curriculum course (i.e., the same course appears in multiple sections within the same semester and department), the instructor is permitted – and indeed expected – to teach that session concurrently across all affected sections. The conflict checker recognizes this case and suppresses the violation flag.

H2 – Room Double-Booking Prevention: No room may host more than one session during the same time slot. This restriction applies globally across all sections being solved in a given batch.

H3 – Building-Routing Restrictions: Domain filtering at variable construction time restricts each session's candidate rooms to those within allowed building blocks:

- CS/SE lecture courses: FCSE and Academic Block (AcB).
- Humanities and General Education courses: Burki Building (BB).
- Lab sessions: Specific lab rooms of the corresponding type only.

This domain restriction is hard, ensuring no solver assignment violates GIKI's physical routing policies.

B. Soft Constraints

Soft constraints are brought to life as a scoring function on domain values of the variables before iteration. Values with higher scores (than other values) are tried first, giving them a higher chance of successfully finding a high-quality assignment without the need to backtrack extensively.

S1 – Morning Preference for Junior Semesters: The time slots that start before 12:00 PM will be given a positive scoring bonus. This indicates the pedagogical approach taken at GIKI of having freshman groups taking earlier classes to minimize the schedule conflicts and align to cognitive readiness patterns. The bonus is additive and does not bar subsequent-slot assignments; simply ranks earlier-slot candidates higher within the domain iteration.

S2 – Room Capacity Scoring: Candidate rooms are rated on the basis of the physical capacity of the room and the ideal occupancy of the room:

- Lecture classes: ideal occupancy of 30 students per section.
- Lab sessions: ideal occupancy of 15 students per lab group.

The score for a candidate room r is:

$$\text{score}(r) = -|\text{capacity}(r) - \text{ideal}| \tag{2}$$

Both the rooms of excessive size (wasted space) and those of undersized (infeasible seating) are punished. The capacity can be treated as a soft constraint to allow the solver to assign an over-sized room when there is no better alternative, and to exhaust better fitting alternatives first.

VI. Input and Output Specification

A. Input Files

The system ingests a structured Microsoft Excel (.xlsx) file. The following data tables are encoded across sheets:

TABLE II
INPUT FILE STRUCTURE

Sheet	Contents
Course Sheet	Course code, title, credit hours, type (L/Lab), assigned instructor(s), department, semester.
Room Sheet	Room ID, building block, type (lecture/lab), capacity.
Time Slots Sheet	Slot index, day-of-week, start time, end time.

Pandas library works with all Excel I/O. Credit-hour expansion step is implemented directly after the parsing step: a course with k credit hours is translated into k session variables to be inputted to the solver.

B. Interactive GUI (Tkinter)

The GUI has been written in Python and the standard Tkinter library and it offers:

- **Filter:** Dropdown to choose operating department (BCS, SE, BDS, BES, BAI, etc.).
- **Semester and Section Filters:** Cascading dropdowns of the number of semesters (1–8) and section within the selected department and semester.
- **Day Filter:** View by day (Monday–Friday) or Week.
- **Full Week Merged View:** A grid that shows all five days, sessions across multiple periods are merged on the screen, giving a complete scrolling view.
- **On-the-Fly Refiltering:** The grid is updated with a change in the filter and does not re-computate the schedule.

C. PDF Export (ReportLab)

The export engine uses the ReportLab library to produce a formally styled multi-page timetable PDF. Each page corresponds to one weekday (Monday–Friday), presenting the schedule in tabular format. Exported files are automatically timestamped (e.g., timetable_BCS_2026-04-29_143512.pdf) to prevent overwrites and enable version tracking.

VII. Time Complexity Analysis

A. Worst-Case Complexity of Backtracking

Suppose that there are n session variables in a solve batch and that there is a maximum size d of a domain (candidate (room, slot) pairs) of any variable. Backtracking finds all possible assignments, in the worst case:

$$O(d^n) \tag{3}$$

This is exponential complexity that is inherent to the general CSP NP-hardness [1]. Backtracking would not be practical to find realistic departmental batch (e.g., 15–20 sessions with domain sizes of 30–80 candidate pairs). At best, when every variable is assigned without conflict on the first try the algorithm will terminate in $O(n)$ time. Experimentally, it is known that the average case, with MRV and Forward Checking turned on, is much better than worst case, as confirmed by the sub-2-second solve times in Section X.

B. Effect of MRV and Forward Checking

MRV Effect: With a constant choice of the variable with minimum domain, MRV dramatically decreases the effective branching factor at each node. Most likely failure causing variables are removed in the early stages, before the tree has been constructed. Practically, MRV has been known to cut constraint checks by one to two orders of magnitude on benchmark CSP instances [2].

Forward Checking Effect: Each successful assignment at once eliminates inconsistent values in the domains of future variables. The effective domain size d of the future variables is monotonically decreasing during search as opposed to being constant. This is particularly successful in crowded scheduling problems where instructor and room conflicts spread far and wide.

Combined Effect: With MRV ordering and Forward Checking, observed solve times per section are under 2 seconds even with 20 or more session variables and domain sizes of 50 or more candidates – an astronomical reduction from the worst-case search tree to a manageable exploration.

C. Partitioning Overhead

The partitioned section-by-section strategy introduces a linear overhead factor proportional to the number of sections. Since partitions are solved independently, the overall system runtime is:

$$O(|\Sigma| \times d^{n_{\max}}) \quad \text{where } n_{\max} \leq 20 \tag{4}$$

This decomposed formulation transforms a single exponential problem into a linear chain of bounded-exponent subproblems.

VIII. Space Complexity

A. Domain Storage

Each variable $v \in V$ maintains a domain list of candidate (room, slot) pairs. With n variables in a batch and maximum domain size d, domain storage requires $O(n \cdot d)$ space.

B. Forward Checking Domain Snapshots

To enable backtracking, Forward Checking saves and restores domain snapshots per recursive level. At search depth k, up to k copies of modified domain entries are maintained. The worst-case space overhead of Forward Checking is $O(n^2 \cdot d)$, since maximum depth equals n and each level may contain up to $O(n \cdot d)$ pruned entries.

C. Assignment Store and Call Stack

The current partial assignment A is represented as a dictionary mapping variable identifiers to (room, slot) pairs, requiring $O(n)$ space. The recursive call stack reaches a maximum depth of n frames, each with bounded local variables, contributing $O(n)$ space.

D. Total Space Complexity

The dominant term is:

$$O(n^2 \cdot d) \tag{5}$$

For a batch of 20 variables with domain sizes of 80, this yields approximately 32,000 stored pairs – comfortably within the memory of any current system.

IX. Limitations

A. Large Departmental Loads

The most important practical limitation is in the departments that have an abnormally large number of sections or courses. The number of sections per batch and the overhead associated with repeated invocations of a solver can cause the cumulative departmental solve time to tens of seconds or more. Very large curricula have edge-case batches which may need human intervention in case the solver fails.

B. No Arc Consistency

The latest implementation has used Forward Checking which is a one step lookahead version of constraint propagation. A more aggressive algorithm, Arc Consistency (AC-3), propagates constraints in a more aggressive way, and can prune domains more aggressively. Adding AC-3 would probably cause further reducing backtracking to the expense of increased overhead per node.

C. Soft Constraint Optimality

The implementation of soft constraint is based on the domain ordering (greedy preference) as opposed to global optimization. The system is highly greedy in meeting soft constraint preferences – early variable assignments can preempt later variables to even achieve a better soft-constraint solution. It could enhance the quality of the solutions with weighted objective functions and local search by post-processing (e.g., hill climbing or simulated annealing).

D. Static Input Schema

The system is highly integrated to the particular format of the input Excel files. Alterations in the naming of columns, the structure of sheets or data encoding will necessitate corresponding changes in the parsers. The further enhanced implementation would add a schema-validated data ingestion layer.

E. No Live Constraint Violation Reporting

A failure to assign a session in an overconstrained batch results in system logging the failure but no GUI diagnostic is given to indicate which specific constraints caused the failure. Manual resolution would be quicker with better diagnostics.

TABLE III
Benchmark Results

Metric	Value
Total sessions scheduled	319+
Hard constraint violations	0
Departments covered	BCS, SE, BDS, BES, BAI
Average section solve time	<2 seconds
PDF pages per export	5 (one per weekday)

X. Results

The solver was benchmarked on the complete data of the GIKI Spring 2026 curriculum across the five departments: BCS (Bachelor of Computer Science), SE (Software Engineering), BDS (Bachelor of Data Science), BES (Bachelor of Environmental Sciences), and BAI (Bachelor of Artificial Intelligence).

The solver produced zero teacher double-bookings, zero room double-bookings, and zero building-routing violations in all tested configurations. Shared-curriculum courses were correctly identified and assigned without raising false-positive conflict flags.

Figs. 1–5 show the Tkinter GUI displaying the interactive full-week merged view across all building blocks. Figs. 6–10 show the exported PDF timetable for the BAI department across all five weekdays.

XI. Conclusion

This project shows that classical algorithmic techniques, namely CSP backtracking search with MRV variable ordering and Forward Checking propagation of constraints are adequate to solve realistic university timetabling problems on the scale of a medium-sized engineering institute. The otherwise unsolvable monolithic CSP is reduced by the partitioned section-by-section solving strategy, with auto-bucketed batch sizing, into a series of sub-two-second calculations. The resultant system is able to schedule more than 319 sessions without any violation of the hard constraint, interactive GUI and formal PDF export.

The project is also an example of the difference between hypothesized worst-case complexity ($O(d^n)$) and practical performance which can be achieved through algorithmic intelligence. MRV and Forward Checking cut down on the effective search space by orders of magnitude, and show that constraint propagation is a highly effective tool in structured, dense scheduling problems. The future work needs to look into Arc Consistency propagation, weighted soft-constraint optimization by local search and a more robust data ingestion architecture to overcome the limitations outlined in Section IX.

Acknowledgment

This work was completed as part of CS378 – Design and Analysis of Algorithms, Spring 2026, GIK Institute of Engineering Sciences and Technology.

References

[1] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco, CA, USA: W. H. Freeman, 1979.

[2] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020, pp. 202–233.

[3] E. K. Burke, K. Elliman, and R. Weare, "A university timetabling system based on graph colouring and constraint manipulation," *J. Res. Comput. Educ.*, vol. 27, no. 1, pp. 1–18, 1994.

[4] R. Qu, E. K. Burke, B. McCollum, L. T. G. Merlot, and S. Y. Lee, "A survey of search methodologies and automated system development for examination timetabling," *J. Sched.*, vol. 12, no. 1, pp. 55–89, Feb. 2009.

[5] F. Rossi, P. van Beek, and T. Walsh, Eds., *Handbook of Constraint Programming*. Amsterdam, The Netherlands: Elsevier, 2006, ch. 4–6.

Appendix

Algorithm 1 BACKTRACK(assignment, variables, domains, constraints)

```
if assignment is complete then
    return assignment
end if
v ← arg min{|D(u)| : u ∈ variables, u ∉ assignment}
if D(v) is empty then
    return FAILURE
end if
ordered_values ← sort(D(v), key = soft_score, desc)
for each val in ordered_values do
    if IS_CONSISTENT(v, val, assignment) then
        assignment[v] ← val
        pruned ← {}
        for each v′ not in assignment do
            for each fval ∈ D(v′) do
                if IS_INCONSISTENT(v′, fval, assignment) then
                    Remove fval from D(v′); record in pruned
                end if
            end for
            if D(v′) is empty then
                Restore all pruned entries
                Remove assignment[v]
                goto next val
            end if
        end for
        result ← BACKTRACK(assignment, variables, domains, constraints)
        if result ≠ FAILURE then
            return result
        end if
        Restore all pruned entries
        Remove assignment[v]
    end if
end for
return FAILURE
```

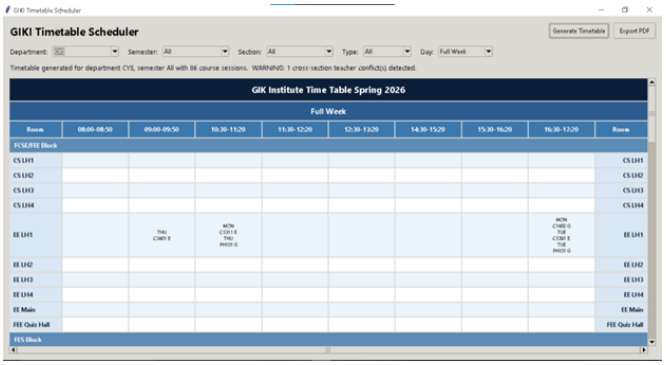

Fig. 1. Tkinter GUI: FCSE/FEE Block room assignments.

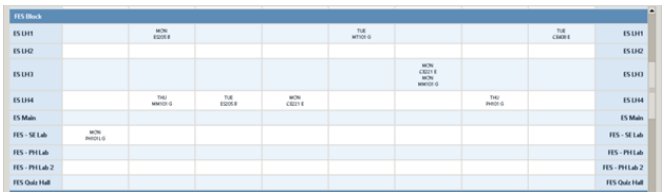

Fig. 2. Tkinter GUI: FES Block room assignments.

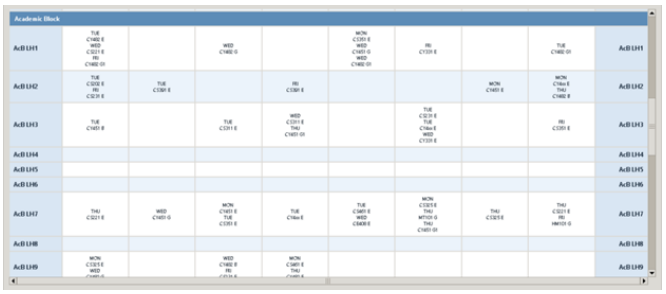

Fig. 3. Tkinter GUI: Academic Block (AcB LH1–LH9).

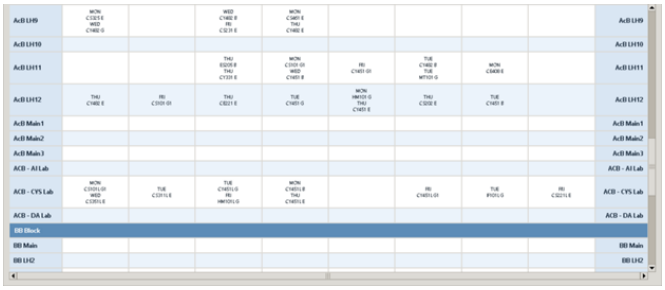

Fig. 4. Tkinter GUI: Academic Block (AcB LH9–LH12) and BB Block.

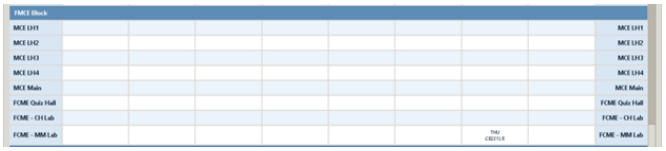

Fig. 5. Tkinter GUI: FMCE Block room assignments.

GIK Institute Time Table Spring 2026 — Full Week

BAI | Semester: All | Monday

Room	08:00-08:50	09:00-09:50	10:30-11:20	11:30-12:20	12:30-13:20	14:30-15:20	15:30-16:20	16:30-17:20	Room
FCSE/FEE Block									
FES Block									
Academic Block									
AcB LH3	AI311 A								AcB LH3
AcB LH7		CS101 F			AI423 A				AcB LH7
AcB LH11	CS478 A								AcB LH11
AcB LH12		AI4xx A							AcB LH12
ACB - CYS Lab	CS311L A IF101L F					HM101L F		AI201L A	ACB - CYS Lab
BB Block									
FME Block									
FMCE Block									
MCE LH2							MM101 F		MCE LH2
FCME - MM Lab		CE413L A							FCME - MM Lab
Other									

Fig. 6. PDF export: BAI timetable – Monday.

GIK Institute Time Table Spring 2026 — Full Week

BAI | Semester: All | Tuesday

Room	08:00-08:50	09:00-09:50	10:30-11:20	11:30-12:20	12:30-13:20	14:30-15:20	15:30-16:20	16:30-17:20	Room
FCSE/FEE Block									
EE LH1					CS391 A		AI321 A PH101 F		EE LH1
FES Block									
Academic Block									
AcB LH3				MT101 F				AI311 A	AcB LH3
AcB LH9	AI321 A				CS417 A				AcB LH9
AcB LH11	CS231 A								AcB LH11
ACB - CYS Lab			AI321L A CS221L A		AI406L A				ACB - CYS Lab
BB Block									
FME Block									
FMCE Block									
MCE LH1					PH101 F				MCE LH1
MCE LH3			MT101 F						MCE LH3
Other									

Fig. 7. PDF export: BAI timetable – Tuesday.

GIK Institute Time Table Spring 2026 — Full Week

BAI | Semester: All | Wednesday

Room	08:00-08:50	09:00-09:50	10:30-11:20	11:30-12:20	12:30-13:20	14:30-15:20	15:30-16:20	16:30-17:20	Room
FCSE/FEE Block									
EE LH1						CS417 A			EE LH1
FES Block									
ES LH4				PH101 F					ES LH4
Academic Block									
AcB LH1							AI423 A		AcB LH1
AcB LH3	CS311 A		CS481 A						AcB LH3
AcB LH7								CS481 A	AcB LH7
AcB LH11			AI321 A		CS231 A		CS391 A		AcB LH11
AcB LH12								CS221 A	AcB LH12
ACB - CYS Lab			CE221L A2			CS101L F			ACB - CYS Lab
BB Block									
FME Block									
FMCE Block									
MCE LH1					CE221 A				MCE LH1
Other									

Fig. 8. PDF export: BAI timetable – Wednesday.

GIK Institute Time Table Spring 2026 — Full Week

BAI | Semester: All | Thursday

Room	08:00-08:50	09:00-09:50	10:30-11:20	11:30-12:20	12:30-13:20	14:30-15:20	15:30-16:20	16:30-17:20	Room
FCSE/FEE Block									
EE LH1							CE413 A		EE LH1
FES Block									
FES - SE Lab							PH101L F		FES - SE Lab
Academic Block									
AcB LH2			CS202 A						AcB LH2
AcB LH3		CS417 A							AcB LH3
AcB LH9	CS325 A			MT101 F					AcB LH9
AcB LH11		CS231 A							AcB LH11
AcB LH12	MM101 F						CS221 A		AcB LH12
BB Block									
FME Block									
FMCE Block									
MCE LH1					CE221 A				MCE LH1
MCE LH4			CE413 A						MCE LH4
Other									

Fig. 9. PDF export: BAI timetable – Thursday.

GIK Institute Time Table Spring 2026 — Full Week

BAI | Semester: All | Friday

Room	08:00-08:50	09:00-09:50	10:30-11:20	11:30-12:20	12:30-13:20	14:30-15:20	15:30-16:20	16:30-17:20	Room
FCSE/FEE Block									
EE LH1	CS478 A				CS478 A			CS202 A	EE LH1
FES Block									
Academic Block									
AcB LH1		AI423 A	CS325 A						AcB LH1
AcB LH2				CS481 A					AcB LH2
AcB LH7					HM101 F	CS221 A			AcB LH7
AcB LH9			AI4xx A	CS311 A				AI4xx A	AcB LH9
AcB LH12	CS391 A	CS325 A CS101 F		CE413 A HM101 F	CS311 A				AcB LH12
BB Block									
FME Block									
FMCE Block									
MCE LH4					CE221 A				MCE LH4
FCME - CH Lab							CE221L A1		FCME - CH Lab
Other									

Fig. 10. PDF export: BAI timetable – Friday.